Campus Smart Resource Allocation and Inventory Management

Project Documentation

TEAM:

SPOORTHI T PES1UG24CS464

SRISHTI RAO PES1UG24CS470

Prepared from the project source files and database scripts.

Documentation date: 06 November 2026

1. Objectives

The project is intended to help campus departments manage shared equipment and consumable supplies across multiple locations.

Objectives:

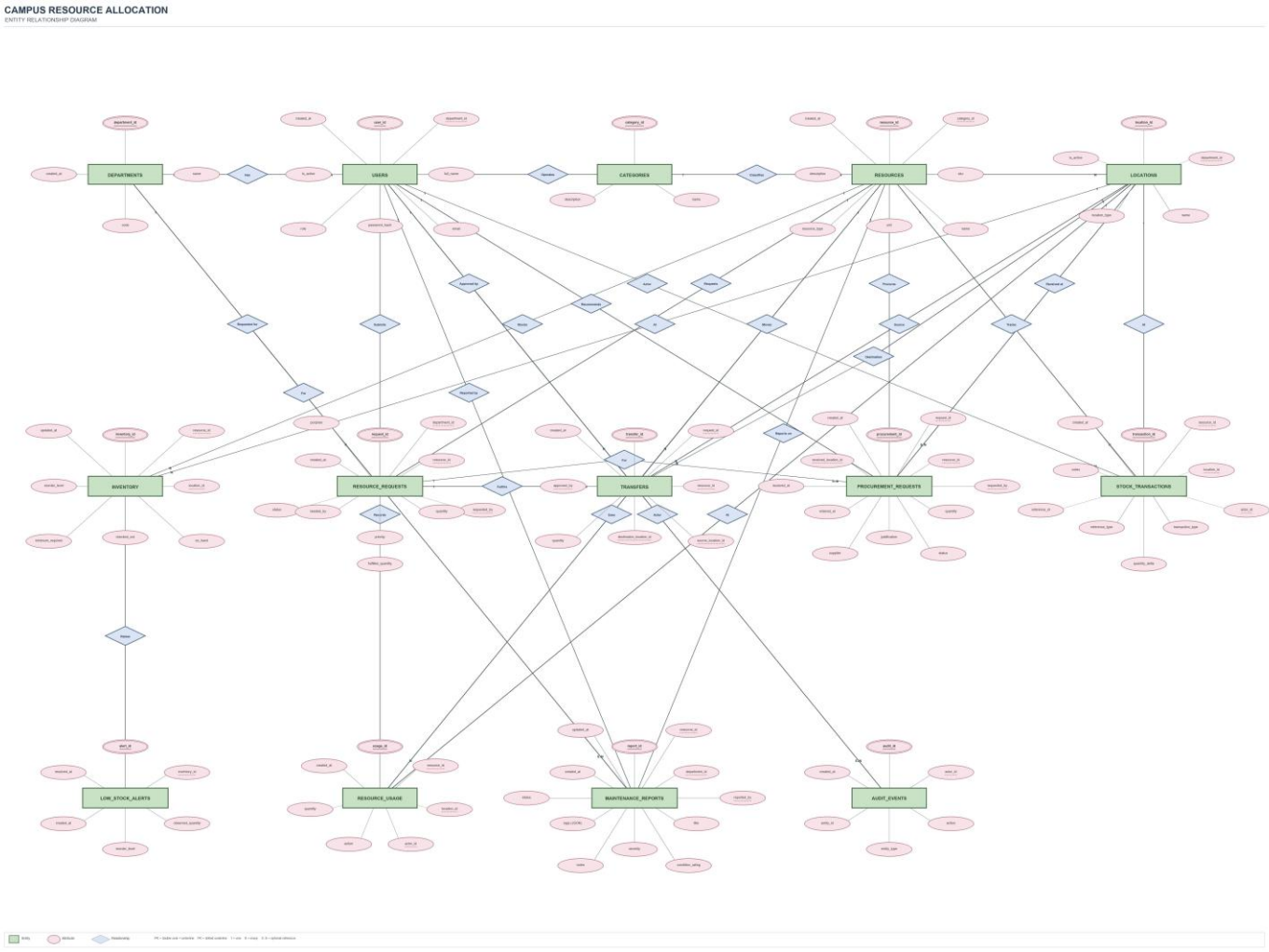
• Show resource quantities and stock levels at campus locations.
• Let staff and students submit resource requests.
• Help administrators reallocate safe surplus while protecting minimum stock.
• Track procurement for quantities that cannot be fulfilled from existing stock.
• Keep a record of inventory changes and maintenance condition reports.

2. Functional requirements

No.	Requirement
1	Users sign in with an email and password. Sessions use a signed, HTTP-only cookie.
2	Authenticated users can view resource and location stock information.
3	Staff and students can create requests; requests include resource, quantity, priority, purpose, and optional due date.
4	Users can view requests allowed for their role; eligible unfulfilled requests can be withdrawn.
5	Administrators can view safe reallocation recommendations for open requests.
6	Administrators can approve transfers; the database procedure protects each source location's minimum stock.
7	Administrators can track procurement recommendations, orders, and receipts.
8	Staff and students can submit maintenance reports; administrators can update report status.
9	The database records inventory, transfer, procurement, and audit history.

The source snapshot inspected for this document contains API handlers and database scripts, but no user-interface page or component files. Features that require a visible UI could not be verified.

3. ER diagram


CAMPUS RESOURCE ALLOCATION
ENTITY RELATIONSHIP DIAGRAM
DEPARTMENTS
USERS
CATEGORIES
RESOURCES
LOCATIONS
INVENTORY
RESOURCE_REQUESTS
TRANSFERS
PROCUREMENT_REQUESTS
STOCK_TRANSACTIONS
LOW_STOCK_ALERTS
RESOURCE_USAGE
MAINTENANCE_REPORTS
AUDIT_EVENTS

4. Relational schema

Space reserved for the project relational schema. Insert the schema here.

Relational schema space

5. Database choice and justification

Database	Use	Reason
MySQL 8	Users, departments, resources, inventory, requests, transfers, usage, procurement, and audit records.	These records have defined relationships and require foreign keys, constraints, transactions, and reliable updates.
MongoDB 6	Maintenance condition reports in the maintenance_reports collection.	Reports have structured fields and a flexible tags list; MongoDB validation and indexes support this document format.
Vector database	Not used.	The project has no implemented embedding or semantic-search requirement.

The MySQL database is named campus_resources. The MongoDB database is campus_smart_resource_allocation. No credentials or private environment settings are included.

6. Data dictionary / schema description

Table	Purpose and key fields
departments	Department ID, name, unique code, and creation time.
users	User ID, optional department, name, unique email, password hash, role, active flag, and creation time.
categories	Resource category ID, unique name, and optional description.
locations	Location ID, optional department, unique name, location type, and active flag.
resources	Resource ID, category, unique SKU, name, unit, resource type, and description.
inventory	Resource and location IDs; on-hand, checked-out, minimum, and reorder quantities. One row per resource/location pair.
resource_requests	Requester, department, resource, requested and fulfilled quantities, priority, status, due date, and purpose.
transfers	Request/resource IDs, source and destination locations, quantity, approving user, and time.
stock_transactions	Stock event type and signed quantity change, resource, location, optional actor, and reference.
resource_usage	Resource, location, actor, action (checkout/return/consume), quantity, and time.
low_stock_alerts	Inventory row, observed quantity, reorder level, creation time, and optional resolution time.
procurement_requests	Optional request, resource, requester, quantity, status, reason, supplier, and order/receipt details.
audit_events	Optional actor, action, entity type and ID, and event time.
maintenance_reports	Relational report structure with resource, reporter, condition, severity, notes, tags, status, and timestamps; the active reports API stores report documents in MongoDB.

Primary keys identify records. Foreign keys connect related records. Check constraints enforce positive request/transfer quantities, prevent fulfilled quantities from exceeding requested quantities, and prevent checked-out stock from exceeding on-hand stock.

7. SQL / NoSQL / vector database implementation

MySQL

database/schema.sql creates the relational tables, the v_resource_availability view, two low-stock triggers, and the sp_approve_transfer stored procedure. The view calculates campus totals and safe reallocatable stock. The triggers create or resolve low-stock alerts. The transfer procedure locks and validates relevant rows, updates source and destination inventory, records the transfer and stock transactions, updates request fulfilment, writes an audit event, and commits the transaction.

MongoDB

database/mongodb.mongosh configures the maintenance_reports collection with a JSON Schema validator. Required report fields include resource, reporter, title, condition, severity, notes, tags, status, and timestamps. Indexes support newest-first reports and severity/status filtering.

Vector database

No vector database is implemented. There is no vector search or embedding workflow in the inspected source.

Integrity and access

The API uses Zod input validation, parameterized MySQL queries, authenticated sessions, and role checks. Passwords are checked against bcrypt hashes; the document does not reproduce any secret values.

8. Important queries

Inventory by location and safely reallocatable quantity

```
SELECT r.sku, r.name, l.name AS location, i.on_hand,<br/> i.checked_out, i.minimum_required,<br/>
GREATEST(i.on_hand - i.checked_out - i.minimum_required, 0)<br/> AS safely_reallocatable<br/>FROM inventory
i<br/>JOIN resources r ON r.resource_id = i.resource_id<br/>JOIN locations l ON l.location_id =
i.location_id<br/>ORDER BY r.name, l.name;
```

Total stock by department

```
SELECT d.code, d.name, SUM(i.on_hand) AS units_held,<br/> COUNT(DISTINCT i.resource_id) AS
resource_types<br/>FROM departments d<br/>JOIN locations l ON l.department_id = d.department_id<br/>JOIN
inventory i ON i.location_id = l.location_id<br/>GROUP BY d.department_id, d.code, d.name<br/>ORDER BY
units_held DESC;
```

Safe surplus allocation logic

```
available surplus = max(0, on_hand - checked_out - minimum_required)<br/>transfer quantity = min(available
surplus, unfulfilled request)<br/>procurement gap = remaining request after recommended transfers
```

The demo SQL file also contains a HAVING query, a subquery using v_resource_availability, request history, a temporary-table DELETE demonstration, and an EXPLAIN query. database/ddl_demo.sql demonstrates CREATE, ALTER, and DROP on a temporary table.

9. System architecture

Part	Responsibility
Client	Sends requests to the web API. UI source files were not present in the inspected checkout.
Next.js API	Handles authentication, validation, authorization, request workflows, resource reads, recommendations, transfers, procurement, and reports.
MySQL	Stores relational campus and inventory data and runs transactional transfer logic.
MongoDB	Stores maintenance condition report documents.
Shared libraries	Provide MySQL/MongoDB connections, session handling, HTTP error handling, and allocation recommendation logic.

Typical request flow: Client -> Next.js API route -> session/role check and input validation -> MySQL or MongoDB -> JSON response.

Login verifies a bcrypt password hash and issues an eight-hour signed session cookie. Administrator-only routes check the user's role. Transfer approval runs through a MySQL stored procedure.

10. Application screenshots

Space reserved for genuine application screenshots. Add screenshots of the running application here.

Application screenshots space

The checked-out source did not contain page/component files or a running UI, so no screenshots are included in this generated document.

11. Testing results

Check	Result	Details
npm test	PASS	6 tests passed. Tests cover safe surplus, splitting requests, minimum stock protection, zero remaining quantity, invalid quantities, and checked-out stock.
Recommendation tests	PASS	6 tests passed when run directly.
npm run lint	PASS	ESLint completed successfully.
npm run build	FAIL	Build reports missing imports @/lib/date and @/lib/usage-allocation.
Database/integration checks	NOT RUN	Require configured database services and test data.

The production build needs the missing helper modules restored or the imports corrected before it can pass. Database integration checks should use a disposable test database.

12. Conclusion

CampusFlow is designed to improve campus inventory visibility and fulfil resource requests by using safe stock reallocation before procurement. MySQL provides transactional storage for requests, inventory, transfers, and audit history, while MongoDB stores maintenance reports. The source includes allocation tests, but the inspected snapshot is incomplete: the production build has unresolved imports, application UI screenshots are not available, and database integration tests were not run.

The ER diagram, relational schema, and application screenshots sections have been left blank for the project team to complete.